Software Engineering

BCSE301L

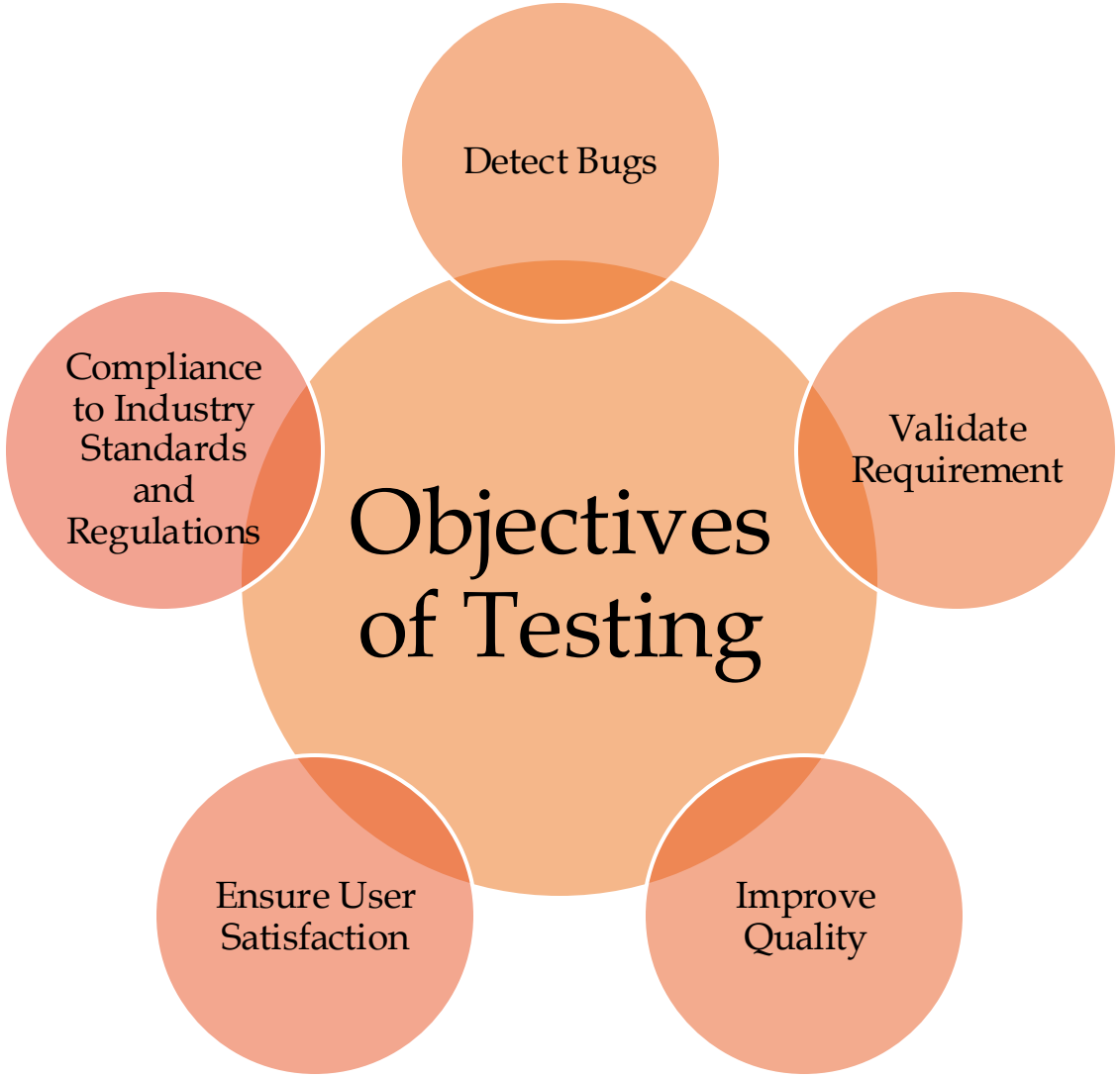
Module 5

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

Software Testing

- Software testing is the process of evaluating a software application to ensure that it meets the specified requirements and is free of defects.
- It involves executing a program or system with the intent of finding errors, verifying its functionality, and ensuring it performs as expected.

Software Testing - Objectives



Software Testing - Objectives

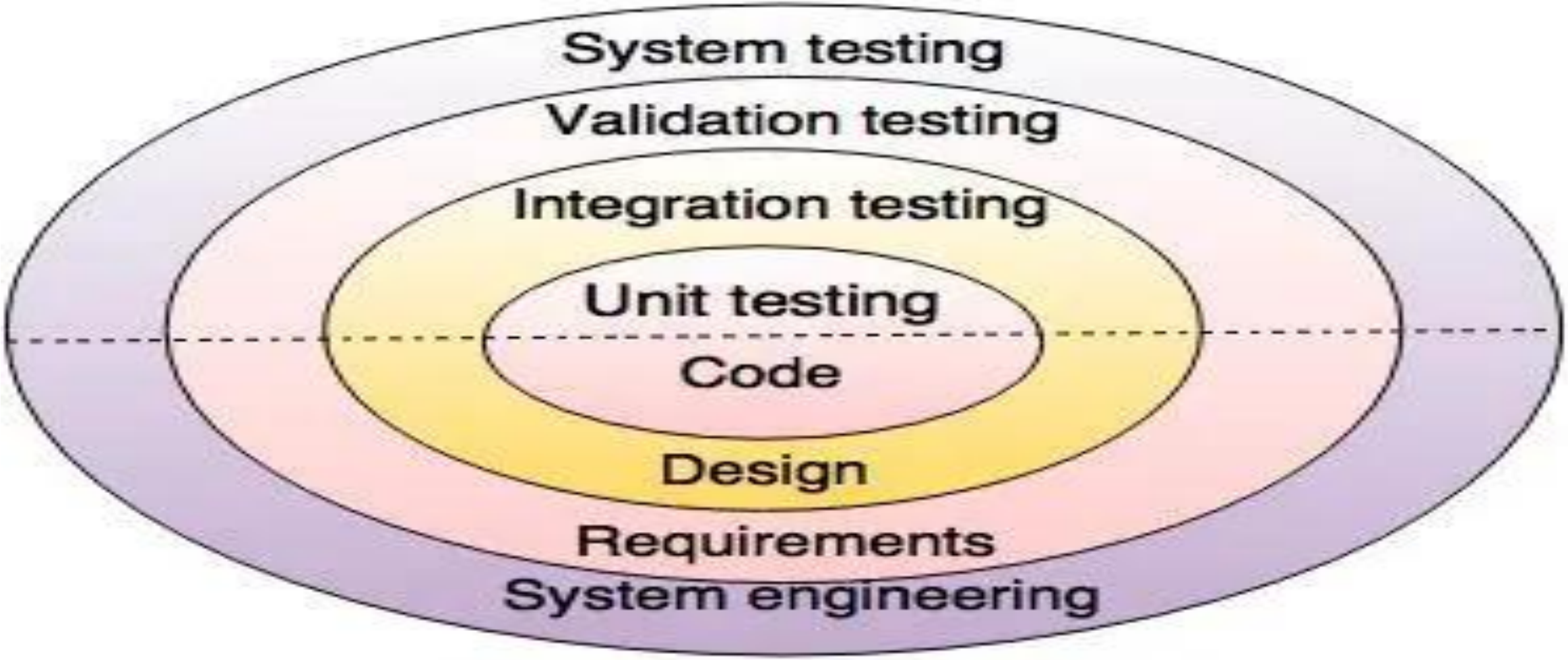


Fig. - Testing Strategy

Software Testing - Objectives

- **Requirement-based Testing**: Requirement-based Testing is a strategy that focuses on verifying whether the software meets the specified requirements. Test cases are designed based on the documented requirements, and the software is evaluated against these requirements to ensure compliance.
- **Functional Testing**: Functional Testing strategy aims to validate the functional aspects of the software by testing individual functions or features. It involves designing test cases to exercise different functionalities and verifying that the software performs as intended.
- **Integration Testing**: Integration testing involves testing the interaction and integration between various software components or modules. It ensures that different components work together seamlessly and accurately to exchange data.
- **System Testing**: System testing is conducted to validate the entire software system. It includes testing all integrated components and subsystems to ensure they function correctly and meet the overall system requirements.
- **Performance Testing**: This strategy evaluates the software's performance under specific conditions, such as load, stress, or scalability. It aims to identify performance bottlenecks, measure response times, and ensure the software can handle expected workloads.

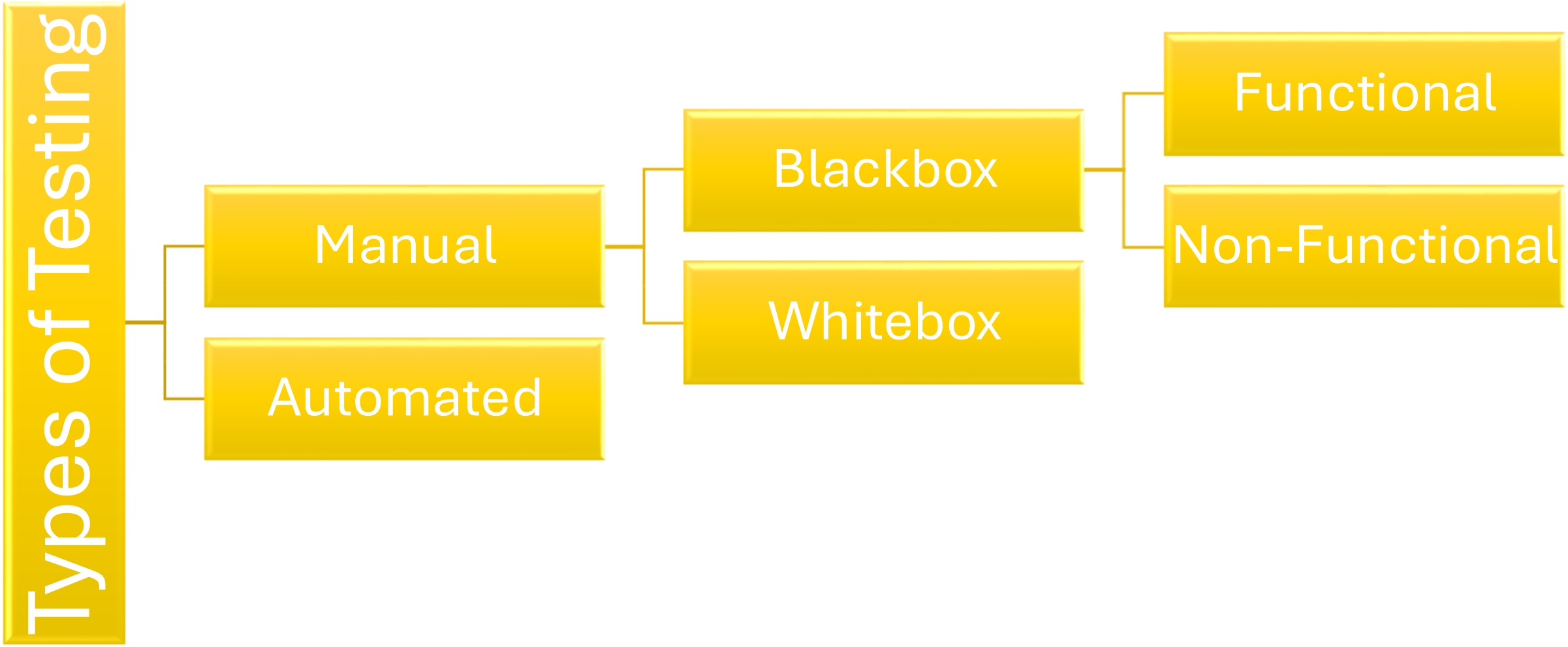
Software Testing - Objectives

- **Regression Testing**: This testing is one of the software testing strategies that focuses on retesting previously tested functionalities to ensure that recent changes or fixes have not introduced new defects or caused any unintended side effects.
- **User Acceptance Testing (UAT):** UAT is performed by end-users or representatives to verify whether the software meets their expectations and requirements. It ensures the software is ready for deployment and use in a real-world environment.
- **Security Testing**: The security Testing strategy aims to identify vulnerabilities and ensure the software's resistance against potential security threats. It includes testing for authentication, authorization, data encryption, and protection against common security attacks.
- **Usability Testing**: Usability testing assesses the software's user-friendliness, ease of use, and overall user experience. It evaluates interface design, navigation, and user interaction to ensure optimal usability.

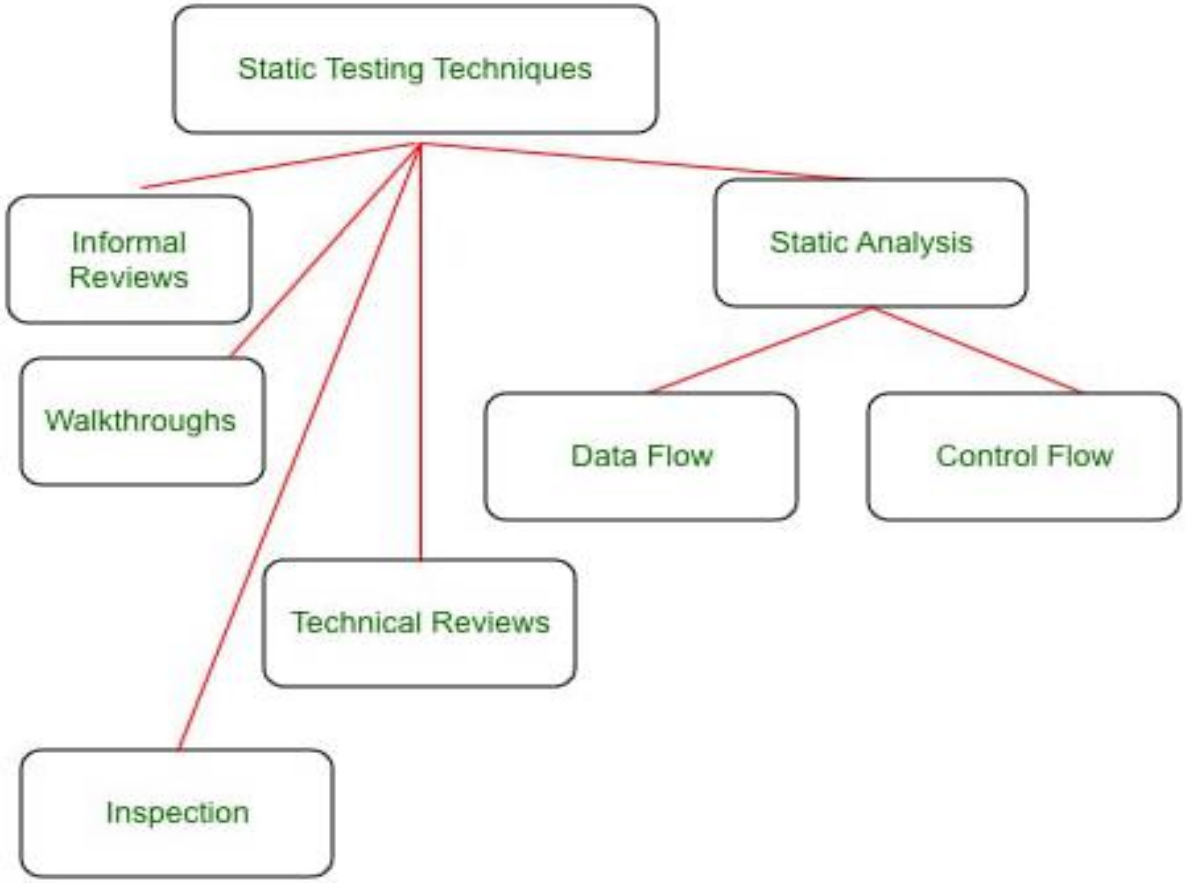
Software Testing - Objectives

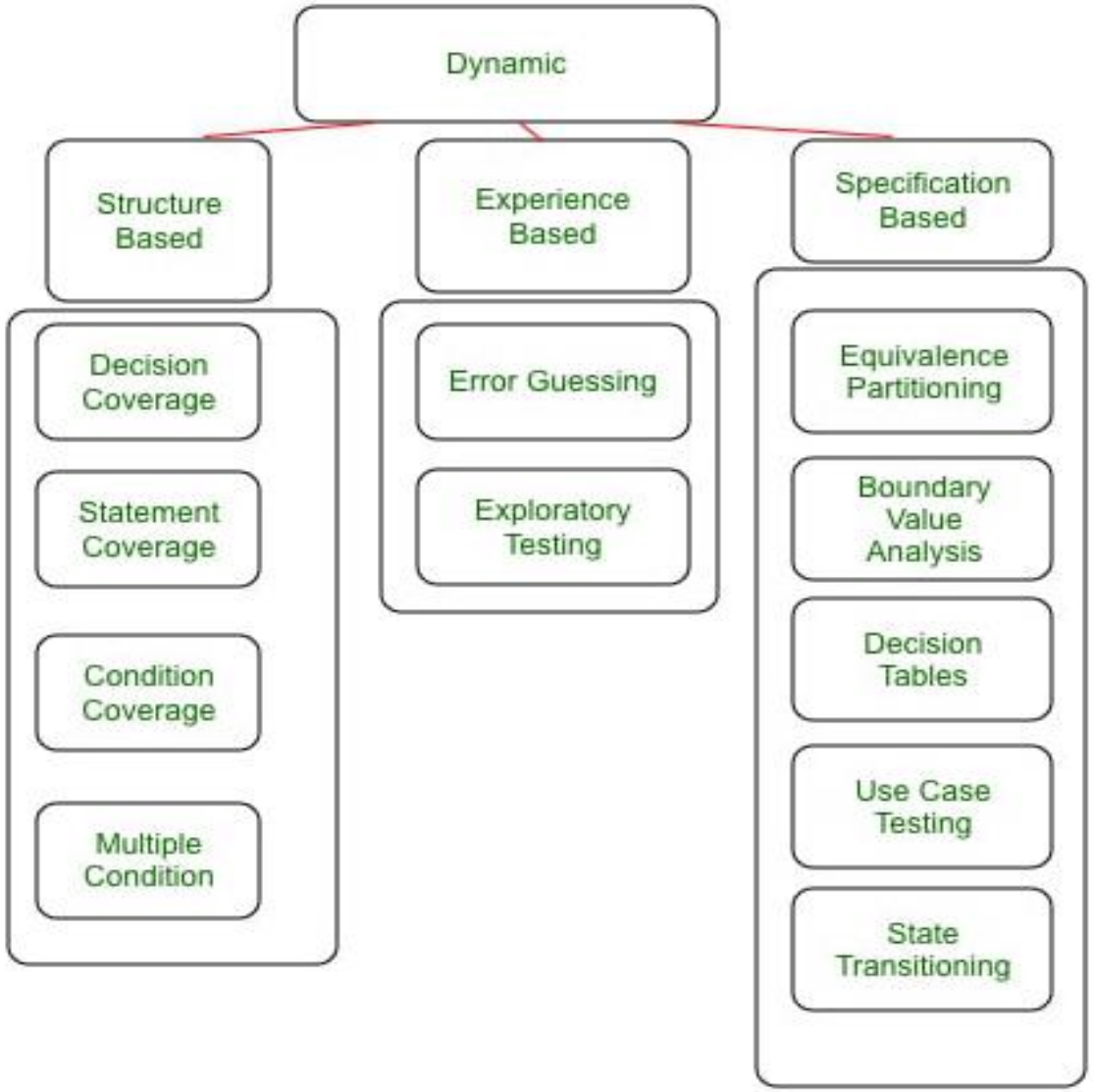
- **Maintenance Testing**: Maintenance testing is performed after software updates or modifications to ensure that the changes have not introduced new defects or adversely affected existing functionalities.

Software Testing - Types



Software Testing - Types





Software Testing - Types

- Manual Testing: Test cases are executed manually without automation.
- Automated Testing: Scripts and tools (e.g., Selenium, JUnit) automate test execution.
- Functional Testing: Verifies specific functions of the application (e.g., unit testing, integration testing).
- Non-Functional Testing: Tests performance, security, usability, etc.
- White Box Testing: Internal code structure is tested.
- Black Box Testing: The tester evaluates functionality without looking at the internal code.

Software Testing – Types

Software Testing – Whitebox Testing

- White box testing is a software testing technique that examines the internal structure, logic, and code of an application.
- It is also known as clear box testing , open box testing , or glass box testing
- **Focus**
  - Tests are designed based on the knowledge of the internal implementation of the system.
  - Testers have access to the source code, algorithms, and architecture of the software.

Software Testing – Whitebox Testing

- **Objective**
  - To ensure that all internal operations function as expected according to the design and specifications.
  - To identify hidden errors, security vulnerabilities, and inefficiencies in the code.

Software Testing – Whitebox Testing

Types of Defects Identified

- Logical errors in the code.
- Broken or poorly structured paths in the code.
- Syntax errors or typos in the code.
- Unreachable or redundant code.

Software Testing – Whitebox – Basis Path Testing

- Basis Path Testing is a structured testing method that identifies a set of basis paths (independent paths) in the program's control flow graph.
- Each path corresponds to a unique sequence of statements and decision outcomes.
- **Objective:**
  - To ensure that every independent path in the program is executed at least once during testing.
  - To achieve 100% path coverage , which guarantees that all logical branches and conditions are tested.

Software Testing – Whitebox – Basis Path Testing

- **Control Flow Graph (CFG) :**
  - A graphical representation of the program's logic, where:
    - Nodes represent statements or blocks of code.
    - Edges represent the flow of control between statements.
    - Decision points (e.g., if, else, switch, loops) create branches in the graph.

Software Testing – Whitebox – Basis Path Testing

- Cyclomatic Complexity
  - A metric used to determine the number of independent paths in the control flow graph.

$$M = E - N + 2P$$

Where:

M = Cyclomatic Complexity

E = Number of edges in the graph

N = Number of nodes in the graph

P = Number of connected components (usually P=1 for a single program).

Cyclomatic Complexity also equals the number of test cases required to achieve full path coverage.

Software Testing – Whitebox – Basis Path Testing

- **Draw the Control Flow Graph (CFG):**
  - Represent the program's logic as a graph with nodes and edges.
  - Identify decision points and their corresponding branches.
- **Calculate Cyclomatic Complexity:**
  - Use the formula $M=E-N+2$ or count the number of decision points (D) and add 1: $M=D+1$
- **Identify Independent Paths:**
  - Determine the minimum set of paths that cover all edges and nodes in the CFG.
  - Each path should include at least one new edge not covered by previous paths.

Software Testing – Whitebox – Basis Path Testing

- **Design Test Cases:**
  - Create test cases to execute each independent path.
  - Ensure that all conditions and branches are tested.
- **Execute Test Cases:**
  - Run the test cases and verify the program's behavior against expected outcomes.

Software Testing – Whitebox – Basis Path Testing

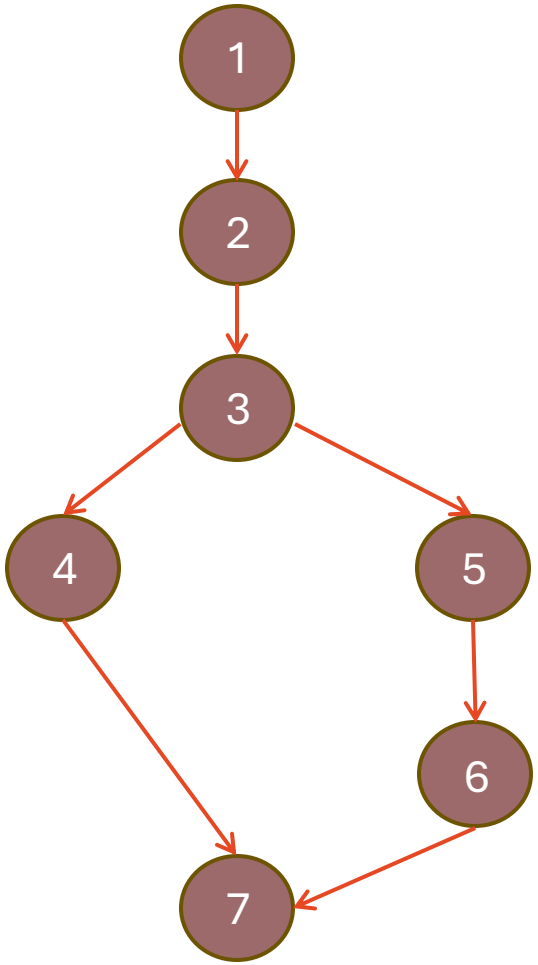
- Example 1

```
1. Read A
2. Read B
3. IF (A > B) THEN
4.    Print "A is greater"
5. ELSE
6.    Print "B is greater"
7. ENDIF
```

Software Testing – Whitebox – Basis Path Testing

- Example 1 – Step 1 – Draw CFG

```
1. Read A
2. Read B
3. IF (A > B) THEN
4.    Print "A is greater"
5. ELSE
6.    Print "B is greater"
7. ENDIF
```



Software Testing – Whitebox – Basis Path Testing

➢Example 1 – **Step 2 – Calculate Cyclomatic Complexity**

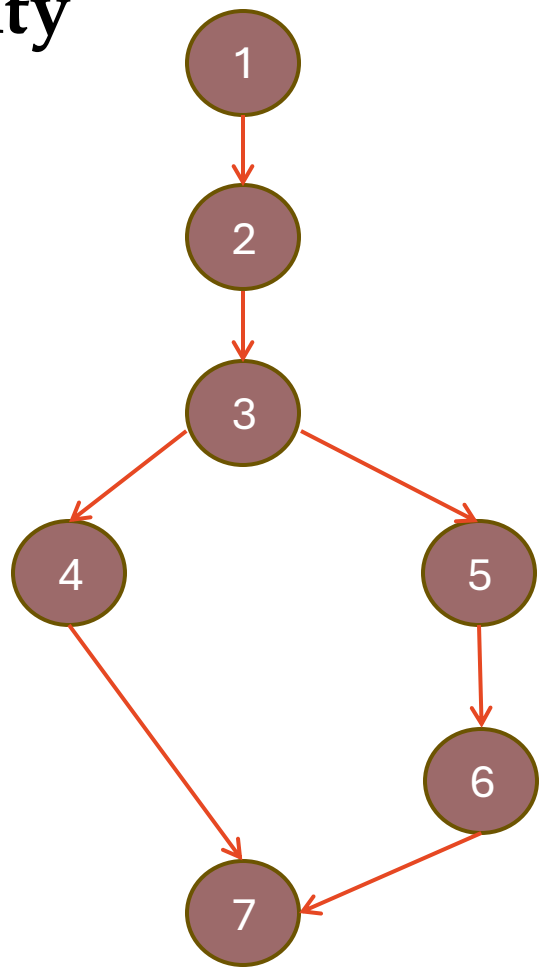
E=7 (edges), N=7 (nodes), P=1 (connected component).

M = E – N + 2

M = 7 – 7 + 2 * 1 = 2

Alternative

M = D + 1 = 1 + 1 = 2



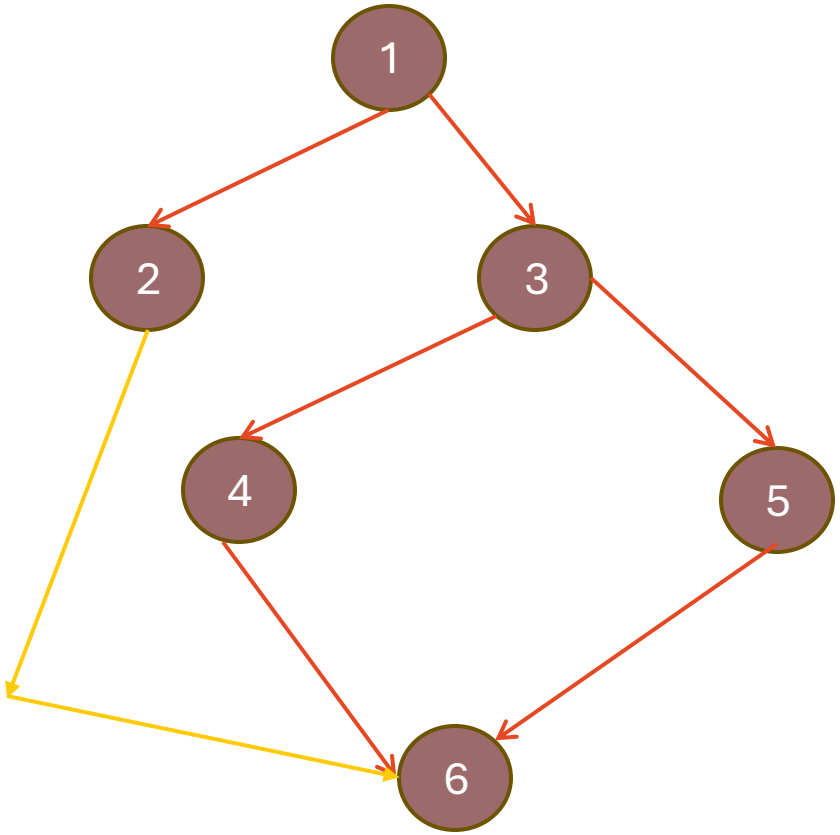
Software Testing – Whitebox – Basis Path Testing

➢Example 2 –

```
{
1. If (a < b)
2.    F1();
else
{
3.    If (a < c)
4.       F2();
   else
5.       F3();
   }
6. }
```

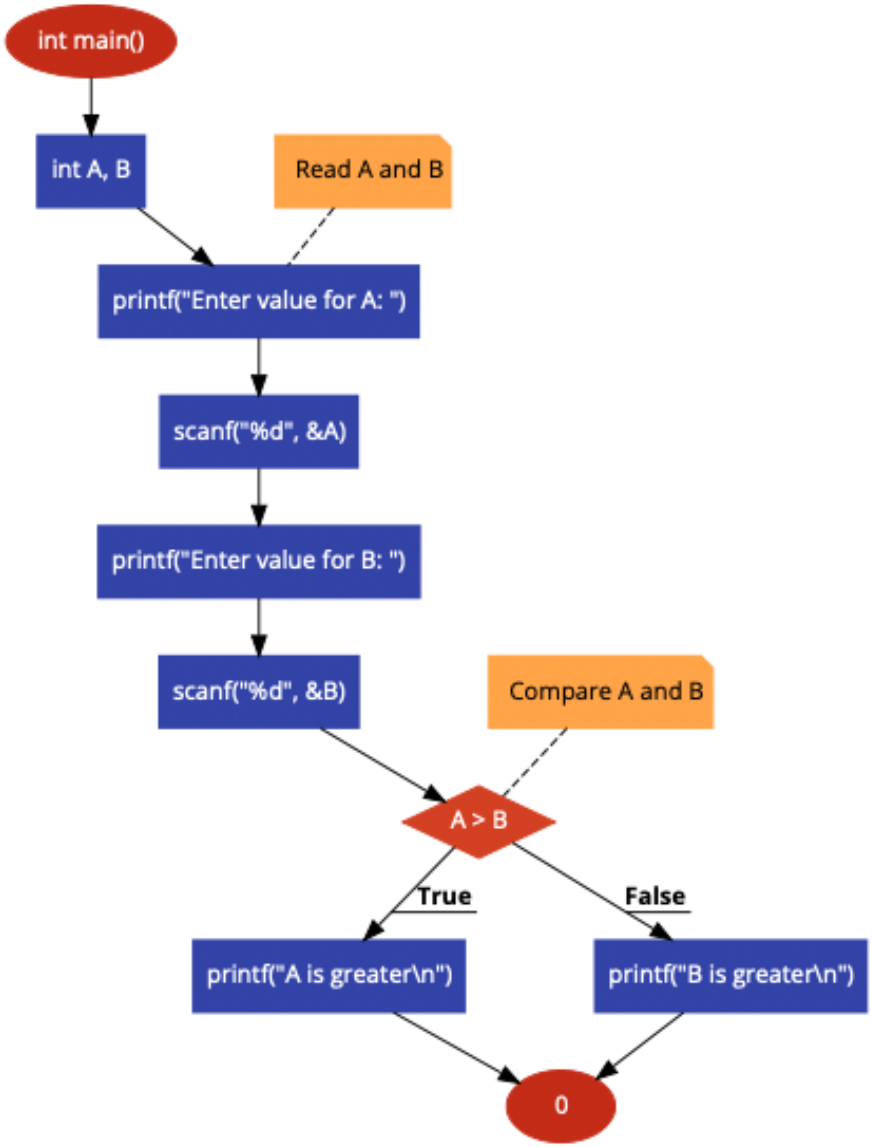
Software Testing – Whitebox – Basis Path Testing

- Example 2 – complexity
- N = 6
- E = 7
- M = E – N + 2
- M = 7 – 6 + 2 = 3



Software Testing – Whitebox – Basis Path Testing

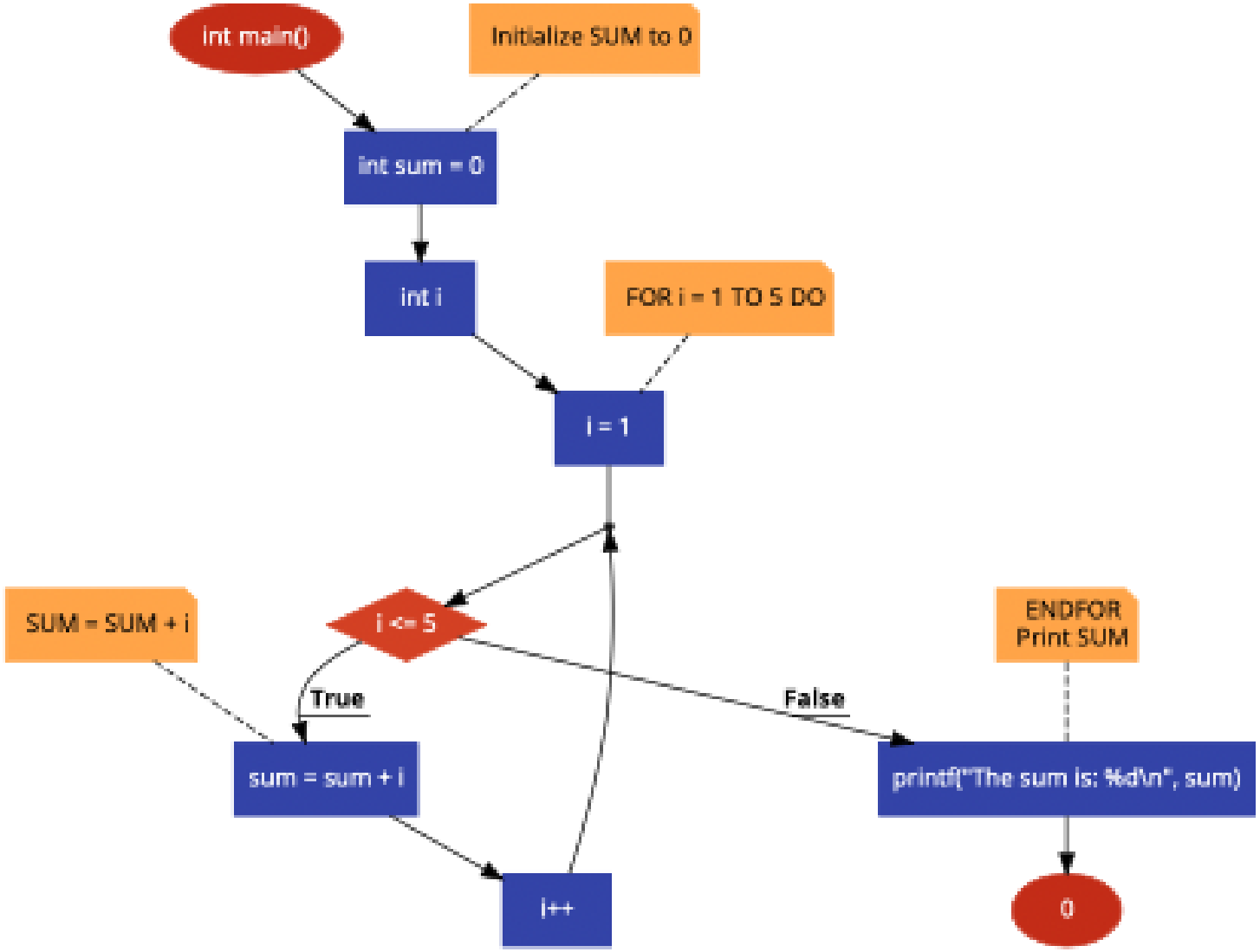
```c
#include <stdio.h>

int main() {
    int A, B;

    // Read A and B
    printf("Enter value for A: ");
    scanf("%d", &A);
    printf("Enter value for B: ");
    scanf("%d", &B);

    // Compare A and B
    if (A > B) {
        printf("A is greater\n");
    } else {
        printf("B is greater\n");
    }

    return 0;
}
```



Software Testing – Whitebox – Basis Path Testing

- Example 2

```
#include <stdio.h>
int main() {
   int sum = 0;  // Initialize SUM to 0
   int i;
   // FOR i = 1 TO 5 DO
   for (i = 1; i <= 5; i++) {
      // SUM = SUM + i
      sum = sum + i;
   }
   // ENDFOR
   // Print SUM
   printf("The sum is: %d\n", sum);
   return 0;
}
```



Software Testing – Whitebox – Control Structure Testing

- **Common Errors in Conditions:**
  - **Boolean Operator Errors** : Incorrect, missing, or extra Boolean operators (e.g., using AND instead of OR).
  - **Boolean Variable Errors** : Using incorrect variables in conditions.
  - **Parenthesis Errors** : Misplaced or missing parentheses leading to incorrect evaluation order.
  - **Relational Operator Errors** : Using incorrect relational operators (e.g., >= instead of >).
  - **Arithmetic Expression Errors** : Errors in expressions used within conditions (e.g., division by zero).

Software Testing – Whitebox Testing

- Statement Coverage
  - To cover as much of the source code as possible.
  - Code coverage is a measure that indicates how much of an application's code contains unit tests that validate its functioning.
  - Using concepts such as statement coverage, branch coverage, and path coverage, it is possible to check how much of an application's logic is really executed and verified by the unit test suite.

$$\text{Statement Coverage} = \frac{\text{Number of Executed Statements}}{\text{Total Number of Statements}} \times 100$$

Software Testing – Whitebox Testing

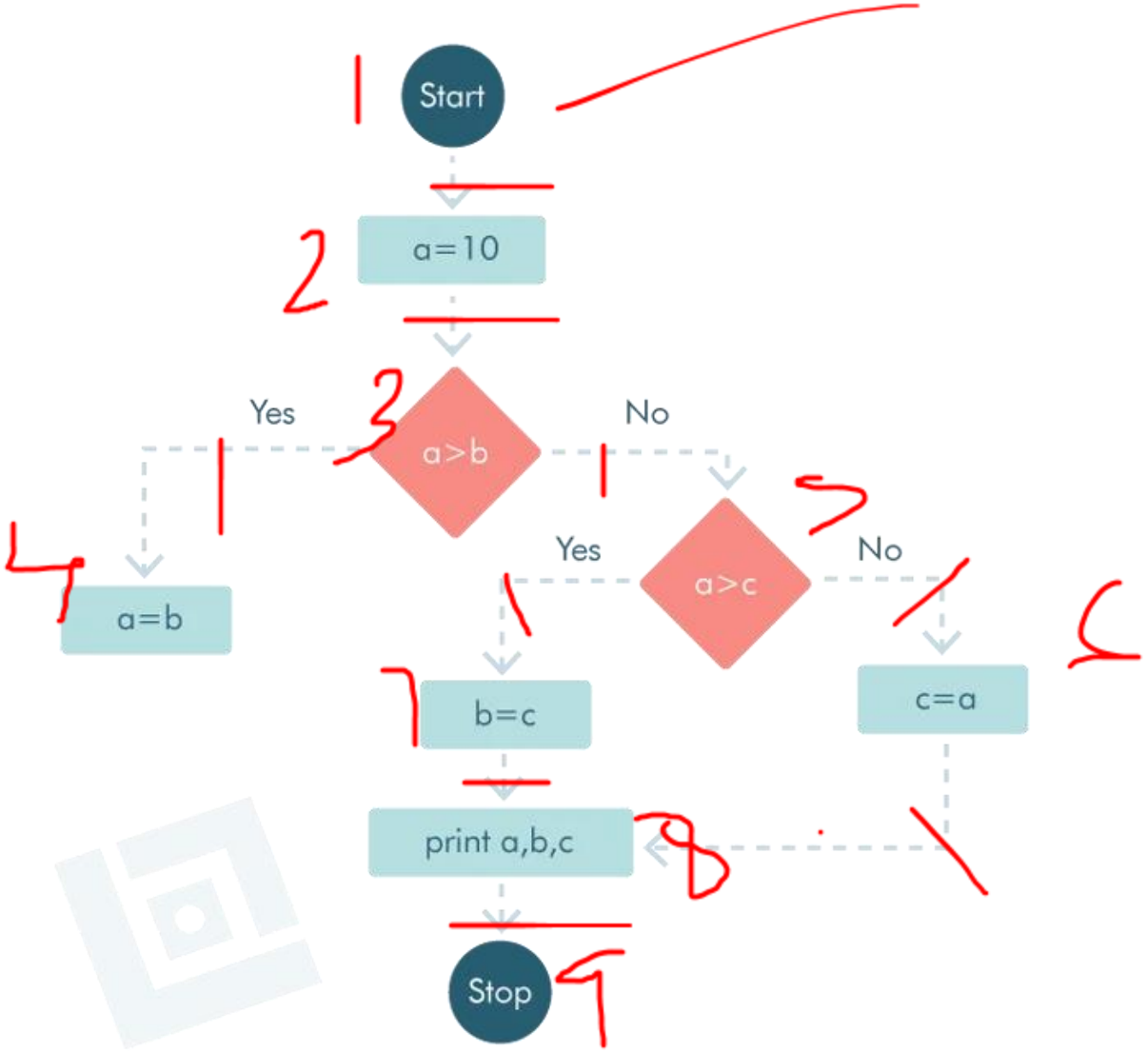
- **Branch Coverage**
- In programming, "branch" is equivalent to, say, an "IF statement" where True and False are the two branches of an IF statement.
- As a result, in Branch coverage, we check if each branch is processed at least once.
- There will be two test conditions in the event of an "IF statement":
- One is used to validate the "true" branch, while the other is used to validate the "false" branch.

$$\text{Branch Coverage} = \frac{\text{Number of Executed Branches}}{\text{Total Number of Branches}}$$

Software Testing – Whitebox Testing

- **Path Coverage**
- Path coverage examines all the paths in each program.
- This is a thorough strategy that assures that all program paths are explored at least once.
- Path coverage is more effective than branch coverage. This method is handy for testing complicated applications.

Software Testing – Whitebox Testing - Cylomatic



Software Testing – Whitebox Testing - Cylomatic

- Method 1
  - E = 10, N = 9, P = 1
  - Hence, V(G) = E - N + 2*P = 10 - 9 + 2 * 1
  - V(G) = 3
- Method 2
  - P = 2
  - Hence, V(G) = P + 1 = 2 + 1
  - V(G) = 3

Software Testing – Whitebox Testing - Cylomatic

- Method 3
  - Number of regions in the control flow graph = 3
  - Hence, V(G) = 3

Software Testing – Whitebox Testing – Graph Matrices

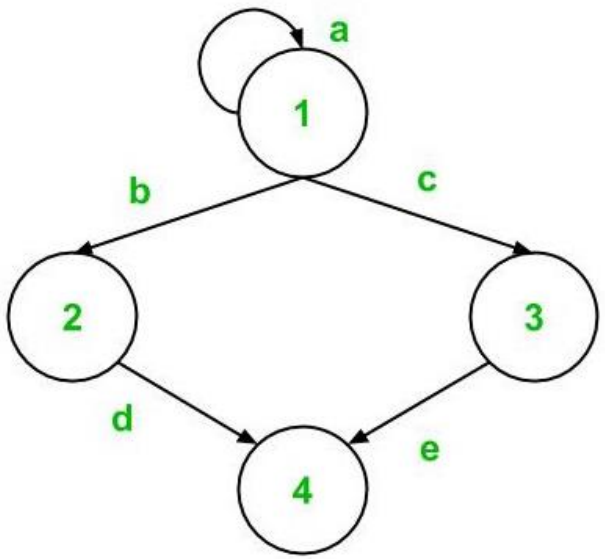
- Flow graph is an effective aid in Path testing
- If the size of the graph increases, manual path tracing becomes difficult and leads to errors, for example a link can be missed or covered twice.
- So, to manage this situation, a software tool can be developed to help basis path testing.
- Graph matrix is a data structure that can assist in developing a tool for automation of path tracing.

Software Testing – Whitebox Testing – Graph Matrices

- A graph matrix is a square matrix whose rows and columns are equal to the number of nodes in the flow graph.
- Each row and column identifies a particular node and matrix entries represent a connection between the nodes.
- Each cell in the matrix can be a direct connection or link between one node to another node
- If there is a connection from node ‘a’ to node ‘b’ then it does not mean that there is a connection from node ‘b’ to ‘a’
- Digits are used to represent the nodes and letter symbols used for edges.

Software Testing – Whitebox Testing – Graph Matrices

- Consider the below graph and represent it in the form of graph matrix.



	1	2	3	4
1	a	b	c	
2				d
3				e
4				

Software Testing – Whitebox Testing – Graph Matrices

- A connection matrix is a tabular representation of a matrix that are defined with link/edge weight.
- If we add link weights to each cell entry, then graph matrix can be used as a powerful tool in testing.
- The link between the two nodes are assigned a link weight which becomes the entry in the cell of matrix. The link weight provides information about control flow.
- In simple form, when a connection exists between two nodes of control flow graph, then the edge weight is 1, otherwise, it is 0. However, 0 is not usually entered in the matrix cells to reduce the complexity.
- A matrix defined with link weights are called connection matrix.

Software Testing – Whitebox Testing – Graph Matrices

- Consider example

	1	2	3	4
1	1	1	1	
2				1
3				1
4				

Software Testing – Whitebox Testing – Graph Matrices

- A connection matrix is used to find the cyclomatic complexity of the control graph.
- Following are the steps to compute the cyclomatic complexity :
  - For each row, count the number of 1s and write it in the end.
  - Subtract 1 from this count for each row (Ignore the row if its count is 0)
  - Add the final count of each row
  - Add 1 to the sum calculated in step 3
  - The final sum in Step 4 is the cyclomatic complexity of the control flow graph

Software Testing – Whitebox Testing – Graph Matrices

- For Example

	1	2	3	4	
1	1	1	1		3 - 1 = 2
2				1	1 - 1 = 0
3				1	1 - 1 = 0
4					Ignore
Cyclomatic complexity = 2 + 0 + 0 + 1 = 3					

Software Testing – Whitebox Testing – Graph Matrices

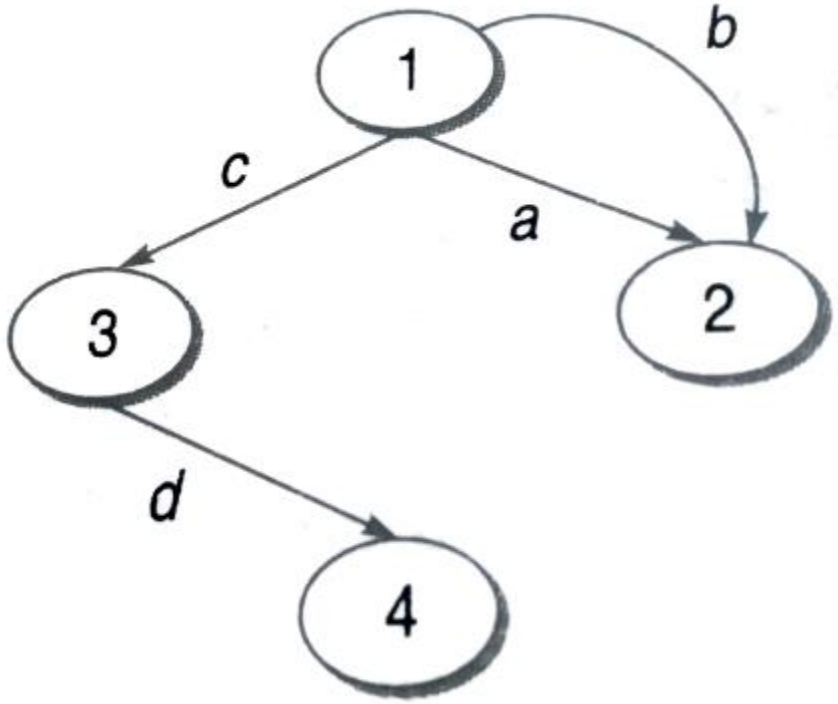
- Graph matrix is used to produce a set of all possible paths between all the nodes.

- For the example 1,

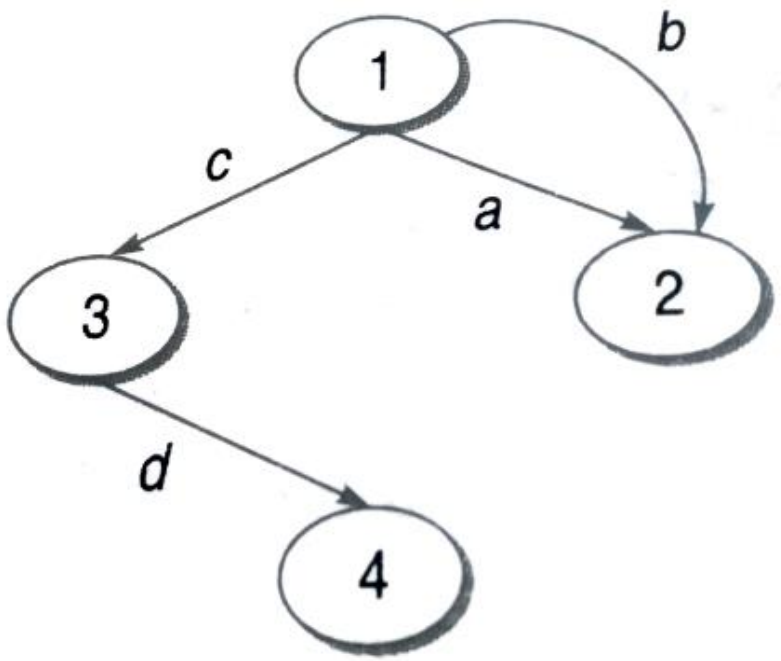
- 1 – 1 // forms a cycle and should be avoided
- 1 – 2 – 4
- 1 – 3 - 4

Software Testing – Whitebox Testing – Graph Matrices

- Consider the below graph and represent it in the form of graph matrix. Draw the connection matrix and find the cyclomatic complexity. List out all the paths.

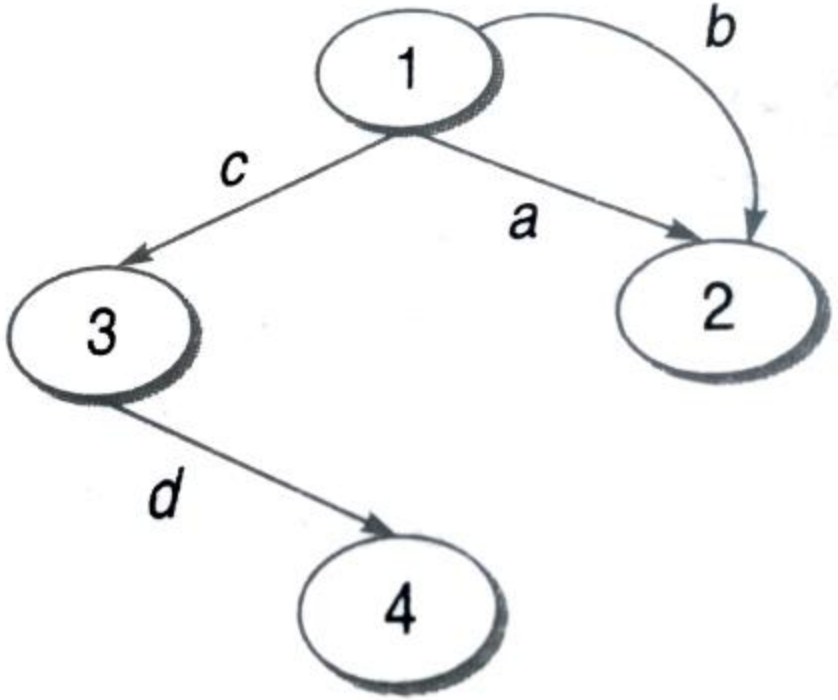


Software Testing – Whitebox Testing – Graph Matrices



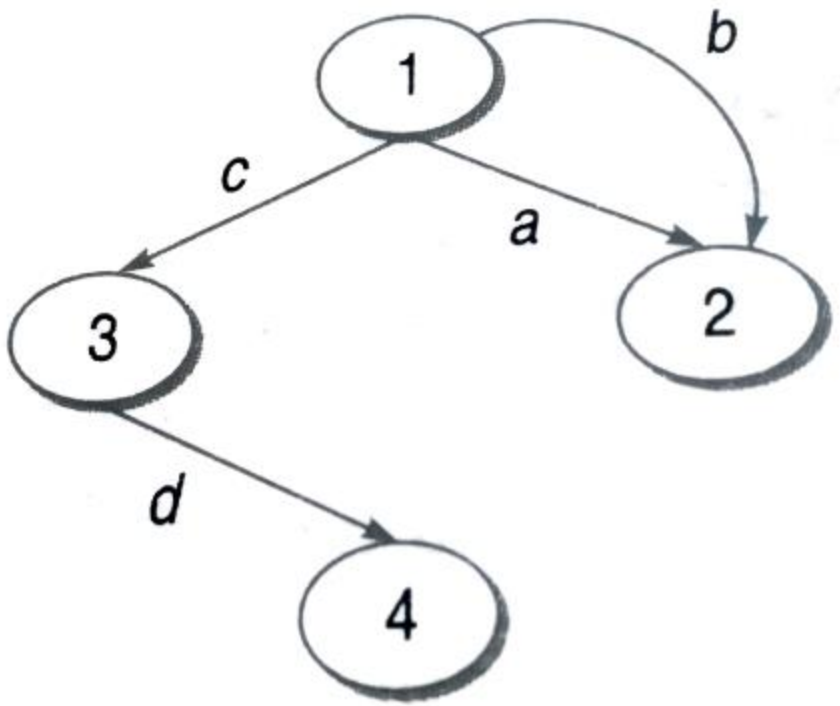
	1	2	3	4
1		a+b	c	
2				
3				d
4				

Software Testing – Whitebox Testing – Graph Matrices



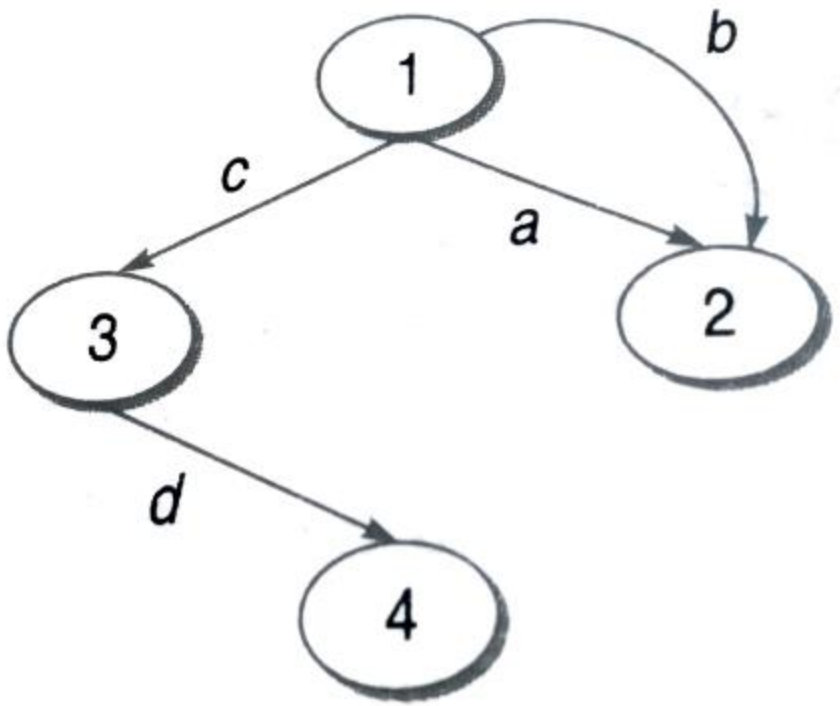
	1	2	3	4
1		1	1	
2				
3				1
4				

Software Testing – Whitebox Testing – Graph Matrices



	1	2	3	4	
1		1	1		2 − 1 = 1
2					
3				1	1 − 1 = 0
4					
	Cyclomatic number = 1+1 = 2				

Software Testing – Whitebox Testing – Graph Matrices



List of Paths

1 – 2

1 – 3 - 4

Software Testing – Blackbox Testing

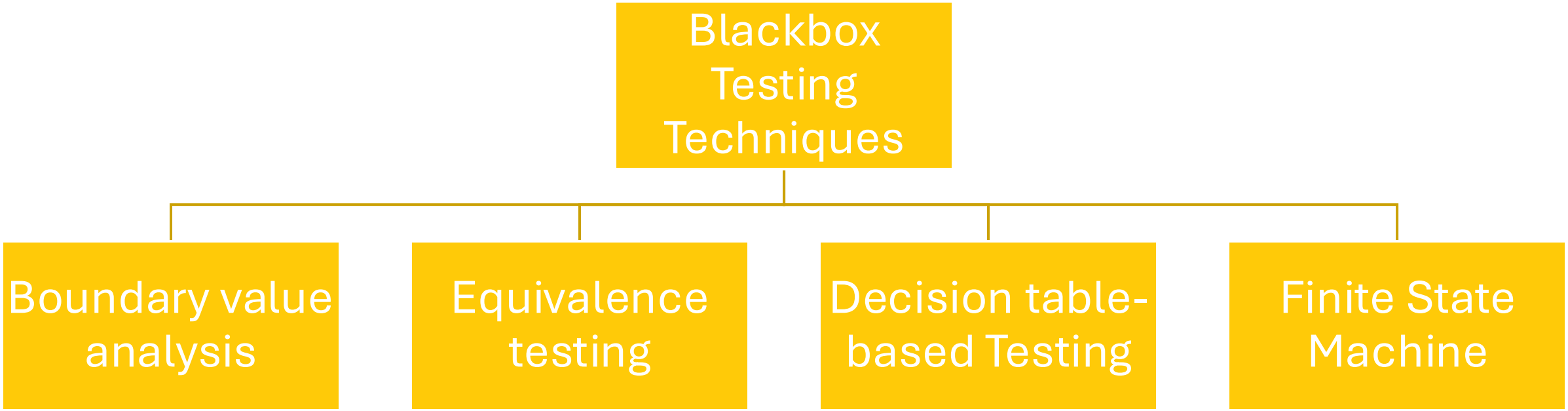
- The focus of Black Box Testing is to check the functionality of the system, without knowing much about the internal structure/design of the item that is being tested and compares the input value with the output value.

  - It is also known as behavioral testing.
  - A tester, without knowledge of the internal structures can perform the black box testing.

Software Testing – Blackbox Testing

- Black-box testing will find the errors in the following category:
  - To test the modules independently
  - To test the functional validity of the software
  - To look for interface errors
  - To test the system behaviors and check the performance
  - To test the maximum load or stress on the system.

Software Testing – Blackbox Testing - Techniques



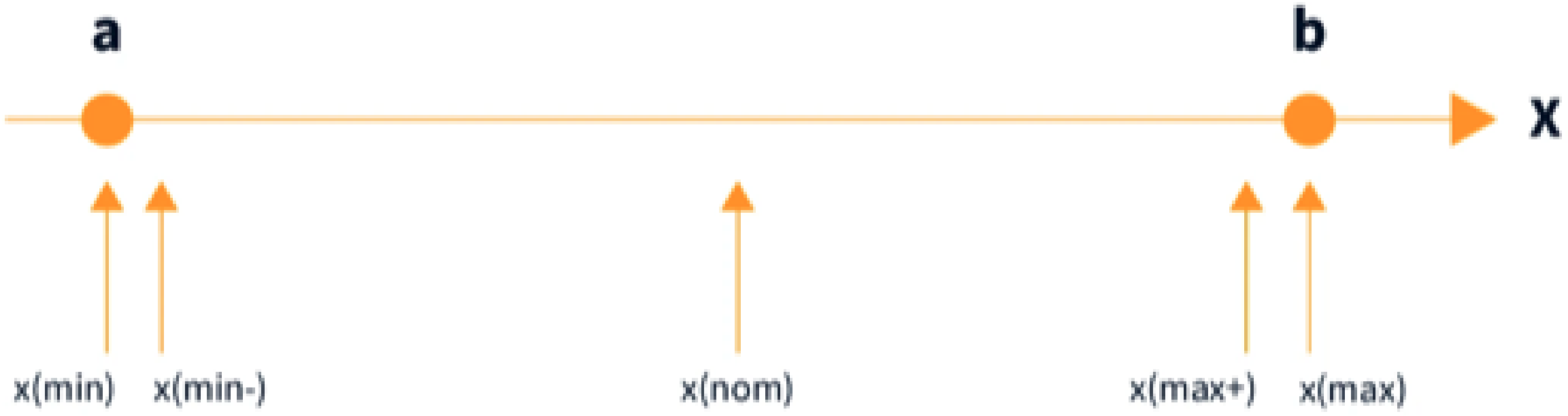
Software Testing – Blackbox Testing – Equivalence Class Partitioning

- Used to reduce the number of test cases while ensuring comprehensive coverage.
- It divides the input data of a software application into different equivalence classes (or groups) such that each member of the class behaves similarly.
- The idea is that if one value in an equivalence class works correctly, all other values in that class should also work correctly

Software Testing – Blackbox Testing - Techniques

- Equivalence Class:
  - A set of inputs that are expected to behave the same way.
  - If one input in the class is tested, it is assumed that all other inputs in the class will behave similarly.
- Valid Equivalence Classes:
  - Inputs that are valid according to the system's requirements.
  - These represent normal or acceptable inputs for which the system should produce correct outputs.
- Invalid Equivalence Classes:
  - Inputs that are invalid according to the system's requirements.
  - These represent edge cases, unexpected inputs, or error conditions that the system should handle gracefully.

Software Testing – Blackbox Testing - Techniques



Software Testing – Blackbox Testing - Techniques

- **Example 1**

You are testing a login system where the password must be between 6 and 12 characters long (inclusive). Any password shorter than 6 characters or longer than 12 characters is invalid.

Using Equivalence Partitioning , identify valid and invalid equivalence classes for this system and create test cases.

Software Testing – Blackbox Testing - Techniques

- Example 1 – **Step 1 - Understand the Input Domain**
  - Valid passwords: Length between 6 and 12 characters.
  - Invalid passwords: Length less than 6 characters or greater than 12 characters.

- **Step 2 - Identify Equivalence Classes**
  - **Valid Equivalence Classes**:
    - Passwords with length 6–12 characters .
  - **Invalid Equivalence Classes**:
    - Passwords with length less than 6 characters .
    - Passwords with length greater than 12 characters .

Software Testing – Blackbox Testing - Techniques

- Example 1 – **Step 3 - Select Representative Values**
  - Valid Class: Choose one value, e.g., "abcdef" (length 6).
  - Invalid Class 1: Choose one value, e.g., "abc" (length 3).
  - Invalid Class 2: Choose one value, e.g., "abcdefghijklmnop" (length 16).

- **Step 4 – Testcases Writing**

Testcase ID	Input (Password)	Expected Output	Status/Class Type
TC1	"abcdef" (length 6)	Valid	Valid
TC2	"abcdefghijkl" (length 12)	Valid	Valid
TC3	"abc" (length 3)	Invalid (too short)	Invalid
TC4	"abcdefghijklmnop" (length 16)	Invalid (too long)	Invalid

Software Testing – Blackbox Testing - Techniques

➢Boundary Value Analysis

- ➢Used to test the boundaries of input domains.
- ➢ It focuses on values at the edges of equivalence classes because errors often occur at these boundaries.
- ➢ For example, if an input range is defined as [6, 12], the boundaries are 6 (minimum valid value) and 12 (maximum valid value).
- ➢Testing just inside, exactly on, and just outside these boundaries helps identify potential issues.

Software Testing – Blackbox Testing - Techniques

Software Testing – Blackbox Testing - Techniques

- Considering the Password Example 1
- **Step 1 - Identify the Input Domain Boundaries**
  - Lower boundary: 6 characters (minimum valid length).
  - Upper boundary: 12 characters (maximum valid length).
- **Step 2: Select Boundary Values**
  - For each boundary, we select three values:
    - **For the lower boundary (6):**
    - Just below: 5 characters (invalid).
    - Exactly on 6 characters (valid).
    - Just above: 7 characters (valid).
    - **For the upper boundary (12):**
    - Just below: 11 characters (valid).
    - Exactly on 12 characters (valid).
    - Just above: 13 characters (invalid).

Software Testing – Blackbox Testing - Techniques

➢Step 3 - Design Test Cases

Testcase ID	Input	Expected Output	Justification
TC1	"abcde" (length 5)	Invalid (too short)	Just below lower boundary
TC2	"abcdef" (length 6)	Valid	Exactly on lower boundary
TC3	"abcdefg" (length 7)	Valid	Just above lower boundary
TC4	"abcdefghijk" (length 11)	Valid	Just below upper boundary
TC5	"abcdefghijkl" (length 12)	Valid	Exactly on upper boundary
TC6	"abcdefghijklm" (length 13)	Invalid (too long)	Just above upper boundary

Software Testing – Blackbox Testing - Techniques

➢ Decision Table Based Testing

- ➢ used to test systems with complex business logic or multiple input conditions.
- ➢ It involves creating a decision table that lists all possible combinations of inputs (conditions) and their corresponding outputs (actions).
- ➢ This technique ensures that all possible scenarios are covered, making it particularly useful for systems where the behavior depends on multiple factors.

Software Testing – Blackbox Testing - Techniques

➢ Decision Table Based Testing

- **Conditions:**
  - These are the input conditions or criteria that affect the system's behavior.
  - For example, in an e-commerce application, conditions might include "Is the user logged in?" or "Is the cart total above $100?”
- **Actions:**
  - These are the outcomes or behaviors of the system based on the combination of conditions.
  - For example, "Apply a 10% discount" or "Redirect to login page."

Software Testing – Blackbox Testing - Techniques

➢ Decision Table Based Testing

- **Rules:**
  - Each rule represents a unique combination of conditions and specifies the corresponding action(s).
  - A decision table typically includes all possible rules to ensure comprehensive coverage.
- **Goal:**
  - To systematically identify and test all possible combinations of inputs and their expected outputs.

Software Testing – Blackbox Testing - Techniques

➢Decision Table Based Testing

➢

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
Username (T/F)	F	T	F	T
Password (T/F)	F	F	T	T
Output (E/H)	E	E	E	H

Software Testing – Blackbox Testing - Techniques

➢State Transition Testing

- ➢ **U**sed to test systems that exhibit different behaviors depending on their current state and the inputs they receive.
- ➢It is particularly useful for systems where the output or behavior depends not only on the current input but also on the sequence of previous inputs
- ➢The goal of state transition testing is to verify that the system transitions between states correctly based on the defined rules and inputs.

Software Testing – Blackbox Testing - Techniques

➢ State Transition Testing

- **State:**
  - A condition or situation in which the system resides at a given time.
  - For example, in an ATM system, possible states might include "Idle," "Card Inserted," "PIN Entered," and "Transaction Complete."
- **Transition:**
  - The movement from one state to another in response to an input or event.
  - For example, inserting a card into an ATM transitions the system from "Idle" to "Card Inserted."

Software Testing – Blackbox Testing - Techniques

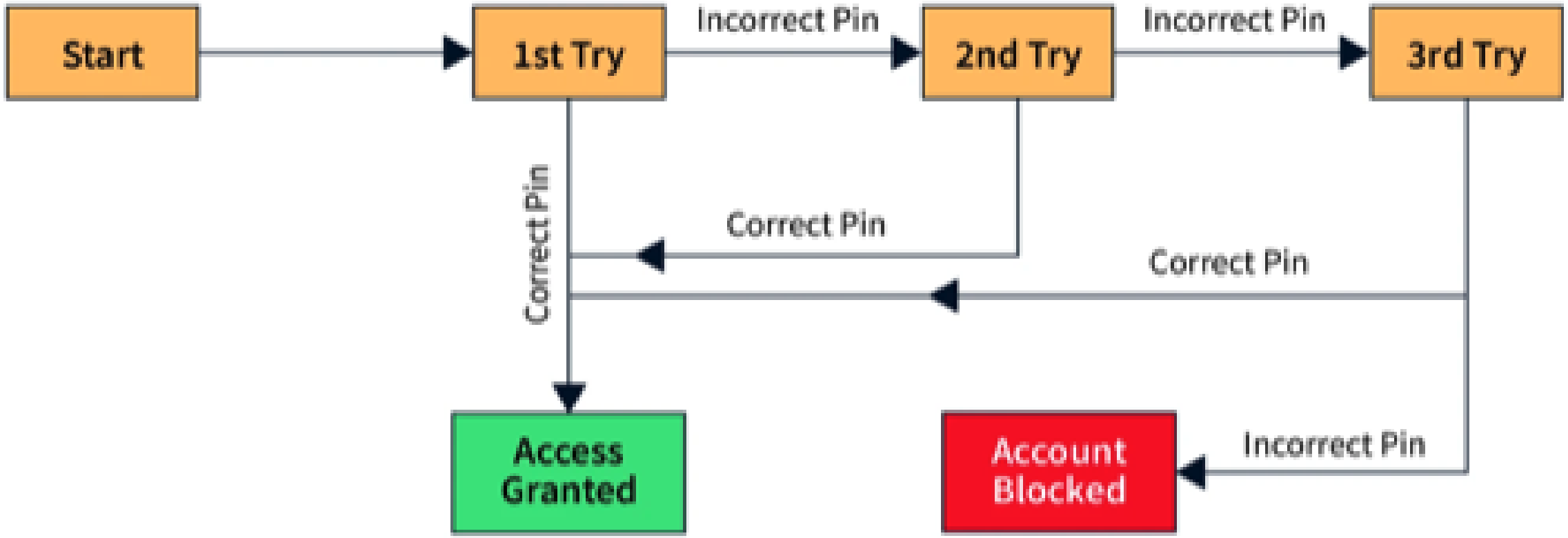
➢ State Transition Testing

- **Event/Input:**
  - An action or trigger that causes a transition between states.
  - For example, entering a PIN or canceling a transaction.

- **Output/Action:**
  - The result or behavior of the system when transitioning between states.
  - For example, displaying a menu after a valid PIN is entered.

Software Testing – Blackbox Testing - Techniques

➢State Transition Testing



Software Testing – Blackbox Testing - Techniques

➢State Transition Testing – Example 1

You are testing a **Login System** with the following states:

Idle: The system is waiting for a user to log in.

Username Entered: The user has entered a username.

Password Entered: The user has entered a password.

Logged In: The user has successfully logged in.

Error: The login attempt failed due to incorrect credentials.

The transitions between states depend on the following events:

Entering a valid username.

Entering a valid password.

Entering an invalid username or password.

Clicking "Cancel."

Software Testing – Blackbox Testing - Techniques

➢State Transition Testing – Example 1

- **Step 1: Create a State Transition Diagram**
- **Step 2: Create a State Transition Table**

Current State	Event /Input	Next State	Output/Action
Idle	Enter Username	Username Entered	Prompt for Password
Username Entered	Enter Valid Password	Logged In	Display Dashboard
Username Entered	Enter Invalid Password	Error	Display Error Message
Error	Retry Login	Idle	Restart Login Process
Logged In	Logout	Idle	Return to Idle State

Software Testing – Blackbox Testing - Techniques

➢State Transition Testing – Example 1

- **Step 1: Create a State Transition Diagram**
- **Step 2: Create a State Transition Table**
- **Step 3: Identify Test Scenarios**

Testcase ID	Intial State	Input/Event	Expected State	Expected Output
TC1	Idle	Enter Username	Username Entered	Prompt for Password
TC2	Username Entered	Enter Valid Password	Logged In	Display Dashboard
TC3	Username Entered	Enter Invalid Password	Error	Display Error Message
TC4	Error	Retry Login	Idle	Restart Login Process
TC5	Logged In	Logout	Idle	Return to Idle State

Software Testing – Regression Testing

- Regression Testing is a type of software testing performed to ensure that recent changes (e.g., bug fixes, new features, or code modifications) do not adversely affect the existing functionality of the application.

- It involves re-running previously executed test cases to verify that the system still behaves as expected after changes.

Software Testing – Regression Testing

- **Key Characteristics:**
  - Purpose: To detect unintended side effects caused by code changes.
  - Scope: Can be applied at various levels (unit, integration, system).
  - Automation: Often automated because regression tests are frequently repeated.

- **When is Regression Testing Performed?**
  - After bug fixes.
  - After adding new features or functionality.
  - After refactoring or optimizing code.
  - Before releasing a new version of the software.

Software Testing – Regression Testing

- **Steps to Perform Regression Testing:**
  - Identify the areas of the application affected by the changes.
  - Select relevant test cases from the existing test suite.
  - Execute the test cases manually or using automation tools.
  - Analyze the results and report any failures.

Software Testing – Regression Testing

- **Advantages:**
  - Ensures stability of the application after changes.
  - Helps maintain software quality over time.
  - Reduces the risk of introducing new defects.

- **Challenges:**
  - Can be time-consuming if done manually.
  - Requires maintaining an up-to-date test suite.
  - Automation tools may need frequent updates to keep pace with changes.

Software Testing – Mutation Testing

- Mutation Testing is a white-box testing technique used to evaluate the quality of test cases.
- It involves making small, deliberate changes (mutations) to the source code and then running the test suite to check if the tests can detect these changes.
- If the tests fail, they are considered effective; if they pass, the tests may need improvement.

Software Testing – Mutation Testing

- **How Mutation Testing Works:**
  - **Mutant Creation**: Introduce small changes (mutations) to the code, such as:
    - Changing operators (e.g., + to -).
    - Modifying conditions (e.g., if (x > 5) to if (x >= 5)).
    - Removing or altering statements.
  - **Test Execution**: Run the test suite on the mutated code.
  - **Result Analysis**:
    - If the tests fail, the mutant is "killed."
    - If the tests pass, the mutant "survives," indicating a potential weakness in the test suite.

Software Testing – Mutation Testing

- **Key Metrics:**
  - **Mutation Score**: The percentage of mutants killed by the test suite. A higher score indicates better test quality.
- **Advantages:**
  - Helps identify weak or incomplete test cases.
  - Encourages thorough testing by ensuring test cases are sensitive to code changes.
  - Provides a quantitative measure of test effectiveness.
- **Challenges:**
  - Computationally expensive due to the large number of mutants.
  - Requires significant effort to analyze results and improve test cases.
  - May produce false positives if mutations are unrealistic.

Software Testing – Object Oriented Testing

- Object-Oriented Testing is a specialized testing approach tailored for object-oriented software systems.
- Since object-oriented programming (OOP) emphasizes concepts like encapsulation, inheritance, and polymorphism, traditional testing techniques may not suffice.
- Object-oriented testing focuses on verifying the correctness of objects, classes, and their interactions.

Software Testing – Object Oriented Testing

- **Class Testing:**
  - Tests individual classes in isolation.
  - Focuses on methods, attributes, and their behavior.
- **Integration Testing:**
  - Tests interactions between classes and objects.
  - Verifies that objects collaborate correctly.
- **Inheritance Testing:**
  - Tests the behavior of derived classes and ensures they inherit and override methods correctly.

Software Testing – Object Oriented Testing

- **Polymorphism Testing:**
  - Tests dynamic binding and ensures that overridden methods behave as expected.
- **Scenario-Based Testing:**
  - Tests real-world use cases involving multiple objects and their interactions.

Software Testing – Object Oriented Testing

- **Challenges in Object-Oriented Testing:**
  - **Encapsulation**: Limited visibility into private methods and attributes makes testing harder.
  - **Inheritance**: Changes in base classes can affect derived classes, requiring extensive retesting.
  - **Polymorphism**: Dynamic method binding complicates predicting which method will execute at runtime.
  - **State Dependencies**: Objects often maintain internal state, making it difficult to isolate tests.

Software Testing – Object Oriented Testing

- **Techniques for Object-Oriented Testing:**
- **Random Testing:**
  - Generates random inputs to test object behavior.
- **Fault-Based Testing:**
  - Introduces faults to identify weaknesses in the design.
- **Scenario-Based Testing:**
  - Simulates real-world scenarios to test object interactions.
- **Use Case Testing:**
  - Tests based on user-defined use cases to ensure the system meets requirements.

Software Testing – Object Oriented Testing

- **Advantages:**
  - Aligns with the principles of OOP, such as modularity and reuse.
  - Facilitates testing of complex systems with multiple interacting objects.
  - Encourages early detection of design flaws.

- **Challenges:**
  - Requires testers to have a deep understanding of object-oriented concepts.
  - Test case design can be complex due to inheritance and polymorphism.
  - Difficult to isolate objects for unit testing due to dependencies.

Software Testing – Testing Web Based Systems

- Web-based systems are applications that run on web browsers and interact with users over the internet.
- Testing these systems is crucial because they must function correctly across various devices, browsers, and network conditions while maintaining performance, security, and usability.
- Testing a web-based system involves multiple types of testing to ensure its functionality, reliability, and user experience.

Software Testing – Testing Web Based Systems

- Functional Testing
- Cross Browser Testing
- Cross Platform testing
- Performance Testing
- Security Testing
- Usability Testing
- Compatibility Testing
- Database Testing
- API Testing
- Accessibility Testing
- Localization and Internalization Testing
- Regression Testing

Software Testing – Blackbox Testing –